

Static Analysis Based Hallucination Detection Reveals Confidence Miscalibration in Transformer-Based Code Comment Generation

Corresponding Author: Shivaansh Sharma

Student at Department of Computer Science and Engineering,
Punjab Engineering College (Deemed to be University),
Chandigarh, India

Email: shivaanshsharma.business@gmail.com

Orcid ID: 0009-0001-4977-7385

Abstract

Transformer-based models such as CodeT5 have demonstrated strong performance in automated code comment generation, often achieving high lexical similarity scores. However, high BLEU scores do not necessarily imply factual correctness or semantic alignment between generated comments and source code. This paper presents a systematic investigation of hallucination behaviour in fine-tuned CodeT5 models and introduces a lightweight static analysis-based verification framework for hallucination detection and mitigation. Experiments conducted on a Python subset of CodeSearchNet demonstrate that although the fine-tuned model achieves a BLEU score of 34.66, approximately 10.2% of generated comments contain entity or structural hallucinations. Regeneration via beam search yields only marginal improvements (9.4%), indicating that decoding strategies alone are insufficient to address hallucination. Furthermore, confidence analysis reveals severe calibration issues: hallucinated outputs exhibit nearly identical softmax probability confidence (0.9981) to non-hallucinated outputs (0.9995). These findings demonstrate that model confidence is unreliable for detecting factual inconsistency and highlight the necessity of external static verification mechanisms for trustworthy code generation systems.

Keywords

Code comment generation, Hallucination detection, CodeT5, Transformer models, Static analysis, Model calibration, Software reliability, Neural code summarization, Confidence miscalibration, Trustworthy AI

1 Introduction

Automated code documentation has become increasingly feasible with transformer-based architectures trained on large-scale code corpora. Models such as CodeT5 leverage pretraining objectives tailored for code understanding and generation, enabling high-quality comment synthesis.

Despite impressive lexical performance, transformer models frequently generate comments that appear syntactically plausible but misrepresent program behaviour. This phenomenon, commonly referred to as hallucination, poses reliability risks in software engineering contexts. Incorrect documentation can mislead developers, propagate misunderstandings, and reduce trust in automated tools.

To the best of our knowledge, this is the first systematic empirical study quantifying hallucination behaviour and confidence miscalibration in transformer-based code comment generation.

While hallucination has been widely studied in natural language generation, its manifestation in code comment generation remains underexplored. In this work, we investigate hallucination characteristics in fine-tuned CodeT5 and propose a static analysis–based detection framework.

We address three research questions:

1. How frequently do hallucinations occur in fine-tuned transformer-based code comment models?
2. Can decoding-based regeneration mitigate hallucination?
3. Does model confidence correlate with hallucination occurrence?

While prior research has examined hallucination in natural language generation, empirical quantification of hallucination behaviour in transformer-based code comment generation remains limited. Furthermore, the relationship between hallucination occurrence and model confidence in code generation tasks has not been systematically studied. This work provides the first empirical evidence of severe confidence miscalibration in neural code comment generation models and demonstrates the necessity of hybrid static–neural verification frameworks.

2 Related Work

Transformer-based models for code representation learning have gained prominence in recent years. CodeT5 introduced identifier-aware pretraining objectives for code understanding and generation tasks. Subsequent works explored code summarization, documentation generation, and code completion.

Hallucination in neural generation has been extensively examined in machine translation and language modelling. Prior research demonstrates that neural models often produce fluent but factually incorrect outputs. Confidence miscalibration in deep neural networks has also been documented, where softmax probabilities overestimate model correctness.

However, hallucination detection in code comment generation remains limited. Existing approaches largely rely on lexical similarity metrics such as BLEU, which fail to capture semantic inconsistencies. Static analysis has traditionally been applied in program verification, but its integration with neural generation systems remains underexplored.

This work bridges transformer-based generation and lightweight static verification for reliability analysis.

3 Methodology

3.1 Model Fine-Tuning

We fine-tuned the Salesforce CodeT5-base model on a Python subset of CodeSearchNet. The training configuration included:

- Training samples: 4,000
- Validation samples: 500
- Test samples: 500
- Epochs: 2
- Mixed-precision training (FP16)
- GPU: NVIDIA RTX 3050

Training converged within two epochs:

- Final training loss: 0.043
- Validation loss stabilized around 0.07

These results indicate effective adaptation without severe overfitting.

3.2 Hallucination Taxonomy

We define two primary hallucination types:

3.2.1 Entity Hallucination

Generated comments reference identifiers not present in the source code.

3.2.2 Structural Hallucination

Generated comments describe control-flow constructs (e.g., loops or conditionals) absent in the source code.

A combined hallucination is detected if either condition is satisfied.

3.3 Static Verification Framework

The proposed verification module operates as follows:

1. Extract code identifiers using regular expressions.
2. Extract identifiers from generated comments.
3. Detect mismatches between identifier sets.
4. Check for inconsistencies in control-flow structure.

This approach avoids AST parsing and remains computationally lightweight.

3.4 Regeneration Strategy

To evaluate mitigation via decoding adjustments, we applied beam-search regeneration (`num_beams = 4`) to outputs flagged as hallucinated.

3.5 Confidence Calibration Analysis

We computed average softmax probability across generated tokens to estimate model confidence. Confidence values were compared between hallucinated and non-hallucinated outputs.

4 Experimental Setup

4.1 Dataset

Experiments were conducted on the Python subset of the CodeSearchNet dataset [20], which contains paired source code functions and corresponding human-written docstrings. To ensure controlled experimentation under limited computational resources, a balanced subset was sampled as follows:

- Training samples: 4,000
- Validation samples: 500
- Test samples: 500

All samples consist of standalone Python functions with associated natural language descriptions. Code and comments were truncated to fixed maximum token lengths to ensure uniform input size.

4.2 Model Architecture

We fine-tuned the Salesforce CodeT5-base model [1], a transformer-based encoder–decoder architecture pre-trained for code understanding and generation tasks.

Model specifications:

- Architecture: Encoder–decoder Transformer
- Parameters: ~220 million
- Pretraining objective: Identifier-aware masked span prediction
- Tokenizer: SentencePiece-based subword tokenizer

CodeT5 leverages both programming language and natural language representations, enabling effective cross-modal generation.

4.3 Training Configuration

Fine-tuning was performed under the following configuration:

- Optimizer: AdamW
- Learning rate: 5×10^{-5}
- Weight decay: 0.01
- Batch size: 8 (per device)
- Maximum input length: 256 tokens
- Maximum output length: 128 tokens
- Number of epochs: 2
- Mixed precision training (FP16) enabled

Training was conducted on:

- GPU: NVIDIA RTX 3050 (8GB VRAM)
- Operating system: Windows
- Framework: PyTorch with HuggingFace Transformers

Total training time was approximately 60 minutes, with convergence observed within two epochs.

Training loss decreased steadily and validation loss stabilized around 0.07, indicating effective adaptation without significant overfitting.

4.4 Decoding Configuration

During evaluation, comment generation was performed using:

- Baseline decoding: Greedy decoding
- Mitigation decoding: Beam search (num_beams = 4)
- Early stopping enabled

Beam search was applied selectively to outputs flagged as hallucinated by the static verification module to evaluate mitigation effectiveness.

4.5 Evaluation Metrics

We evaluated model performance using a combination of lexical and reliability-based metrics:

4.5.1 Lexical Quality

- BLEU score computed using SacreBLEU [21,22]

4.5.2 Hallucination Detection

- Entity hallucination rate

- Structural hallucination rate
- Combined hallucination rate

4.5.3 Calibration Analysis

- Average softmax probability confidence per output
- Confidence comparison between hallucinated and non-hallucinated outputs

Statistical testing for mitigation effectiveness was performed using paired bootstrap resampling (1000 samples).

4.6 Implementation and Reproducibility

All experiments were implemented using:

- Python 3.11
- PyTorch
- HuggingFace Transformers
- SacreBLEU

Random seeds were fixed for reproducibility across runs.

The hallucination detection module relies solely on lightweight regular-expression-based static verification without AST parsing, ensuring computational efficiency and reproducibility.

5 Results

Code Snippet	Generated Comment	Issue Type
def add(a,b): return a+b	"Calculate the sum of a and b."	Correct
def multiply(a,b): return a*b	"Return the product of x and y."	Entity Hallucination
def get_value(x): return x	"Iterate through the list and return the value."	Structural Hallucination

In the second example, the model introduces identifiers (“x” and “y”) not present in the source code. In the third example, the model describes an iteration construct despite the absence of any loop in the program. These cases demonstrate that hallucinations are semantically misleading rather than syntactically malformed.

5.1 Baseline Performance

Despite strong lexical similarity, more than 1 in 10 generated comments contained factual inconsistencies.

Metric	Value
BLEU	34.66
Hallucination Rate	10.2%

5.2 Regeneration Mitigation

Beam-search regeneration produced only marginal reduction in hallucination, indicating that decoding strategies alone do not effectively resolve semantic inconsistencies.

To evaluate whether the observed reduction in hallucination rate under beam-search regeneration is statistically significant, we conducted a paired bootstrap test with 1000 resamples. The reduction from 10.2% to 9.4% was not statistically significant ($p > 0.05$), indicating that decoding strategies alone do not substantially mitigate hallucination.

Metric	Baseline	Beam Regeneration
BLEU	34.66	34.52
Hallucination Rate	10.2%	9.4%

5.3 Confidence Calibration

Category	Average Confidence
Hallucinated	0.9981
Non-Hallucinated	0.9995

The negligible difference in average confidence (0.9981 vs 0.9995) suggests epistemic overconfidence. This phenomenon mirrors calibration issues observed in natural language transformers [14,18], but its implications are more critical in software engineering contexts where incorrect documentation can misguide developers. The results demonstrate that softmax probability cannot serve as a reliable uncertainty proxy in neural code summarization.

6 Discussion

Our findings reveal three critical observations:

1. High BLEU scores do not imply factual correctness.
2. Decoding-based mitigation provides limited improvement.
3. Model confidence does not meaningfully distinguish hallucinated outputs.

These results highlight a reliability gap in transformer-based code comment generation.

Static verification mechanisms provide an interpretable and efficient complementary layer for ensuring factual alignment.

The findings reveal a structural reliability limitation in transformer-based code comment generation. The model is not only prone to hallucination but also exhibits high confidence when incorrect. This misalignment between confidence and correctness indicates that internal probability estimates fail to encode factual uncertainty. Consequently, external verification

layers are essential for deployment in safety-critical or collaborative development environments.

These findings suggest that lexical evaluation metrics alone are insufficient proxies for factual reliability in neural code summarization.

7 Limitations

- Rule-based detection may miss deeper semantic inconsistencies.
- Evaluation limited to Python subset.
- Mitigation does not modify model parameters.
- Confidence estimation based solely on softmax probability.

Future work should explore constrained decoding and training-time calibration strategies.

The rule-based verification approach does not capture deeper semantic contradictions that require full program analysis.

8 Conclusion

This study demonstrates that transformer-based code comment generation models exhibit measurable hallucination behaviour despite strong lexical performance. Decoding adjustments alone are insufficient for mitigation, and softmax confidence fails to distinguish hallucinated outputs. Lightweight static verification provides an effective and interpretable detection mechanism, highlighting the necessity of hybrid reliability frameworks for trustworthy code generation systems.

9 Declarations

Competing Interests

The author declares that there are no competing interests.

Funding Information

No funding was received for this research.

Author Contributions

Shivaansh Sharma conceived the study, designed the methodology, implemented the experiments, performed the analysis, and wrote the manuscript.

Data Availability Statement

The CodeSearchNet dataset used in this study is publicly available. Processed data and experimental scripts generated during the current study are available from the corresponding author on reasonable request.

Research Involving Human and/or Animals

This study does not involve human participants or animals.

Informed Consent

Not applicable.

References

- [1] Wang, Y., Wang, W., Joty, S., Hoi, S.C.H.: CodeT5: Identifier-aware unified pre-trained encoder–decoder models for code understanding and generation. In: EMNLP (2021)
- [2] Feng, Z., Guo, D., Tang, D., et al.: CodeBERT: A pre-trained model for programming and natural languages. In: EMNLP (2020)
- [3] Ahmad, W., Chakraborty, S., Ray, B., Chang, K.-W.: A transformer-based approach for source code summarization. In: ACL (2020)
- [4] Iyer, S., Konstas, I., Cheung, A., Zettlemoyer, L.: Summarizing source code using a neural attention model. In: ACL (2016)
- [5] Allamanis, M., Barr, E.T., Bird, C., Sutton, C.: A survey of machine learning for big code and naturalness. *ACM Computing Surveys* 51(4), 1–37 (2018)
- [6] Guo, D., Ren, S., Lu, S., et al.: GraphCodeBERT: Pre-training code representations with data flow. In: ICLR (2021)
- [7] Chen, M., Tworek, J., Jun, H., et al.: Evaluating large language models trained on code. *arXiv:2107.03374* (2021)
- [8] Roziere, B., Lachaux, M.-A., Chatussot, L., Lample, G.: CodeGen: An open large language model for code generation. *arXiv* (2022)
- [9] Nijkamp, E., Pang, B., Hayashi, H., et al.: CodeGen: Multi-turn program synthesis. *arXiv* (2022)
- [10] Maynez, J., Narayan, S., Bohnet, B., McDonald, R.: On faithfulness and factuality in abstractive summarization. In: ACL (2020)
- [11] Ji, Z., Lee, N., Frieske, R., et al.: Survey of hallucination in natural language generation. *ACM Computing Surveys* 55(12), 1–38 (2023)
- [12] Dziri, N., Kamalloo, E., Clarke, C.: Faithfulness and factuality in abstractive summarization. In: NAACL (2022)
- [13] Rawte, V., Sheth, A., Das, A.: A survey of hallucination in large language models. *IEEE Access* (2023)
- [14] Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: ICML (2017)
- [15] Minderer, M., Djolonga, J., Romijnders, R., et al.: Revisiting the calibration of modern neural networks. In: NeurIPS (2021)

- [16] Lakshminarayanan, B., Pritzel, A., Blundell, C.: Simple and scalable predictive uncertainty estimation using deep ensembles. In: NeurIPS (2017)
- [17] Gal, Y., Ghahramani, Z.: Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: ICML (2016)
- [18] Desai, S., Durrett, G.: Calibration of pre-trained transformers. In: EMNLP (2020)
- [19] Kadavath, S., Conmy, A., Askeel, A., et al.: Language models (mostly) know what they know. arXiv (2022)
- [20] Ren, S., Guo, D., Lu, S., et al.: CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv (2019)
- [21] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: A method for automatic evaluation of machine translation. In: ACL (2002)
- [22] Post, M.: A call for clarity in reporting BLEU scores. In: WMT (2018)
- [23] Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention is all you need. In: NeurIPS (2017)
- [24] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers. In: NAACL (2019)
- [25] Zhang, Y., Sun, S., Galley, M., et al.: DialoGPT: Large-scale generative pre-training for conversational response generation. In: ACL (2020)
- [26] Lin, C.-Y.: ROUGE: A package for automatic evaluation of summaries. In: ACL Workshop (2004)
- [27] Zhou, S., Xu, J., Li, X., et al.: Improving factual consistency of abstractive summarization via constrained decoding. In: ACL (2023)
- [28] Xu, F., Liu, X., Chen, B., et al.: A survey on trustworthy AI in software engineering. IEEE Transactions on Software Engineering (2023)